

搜索功能实现方案

2026年03月14日 v1.0

搜索功能实现方案

文档版本：v1.0

编写日期：2026 年 3 月 14 日

开发负责人：王舟

一、需求分析

1.1 搜索范围

搜索对象	说明	优先级
案件	案件名称、描述、编号	P0
证据文件	文件名、解析文本、标签	P0
事件节点	节点标题、描述、标签	P0
人物	姓名、电话、单位	P0
全文搜索	跨所有类型搜索	P1

1.2 搜索功能

功能	说明	优先级
关键词搜索	单关键词匹配	P0

功能	说明	优先级
高级搜索	多条件组合搜索	P1
全文搜索	SQLite FTS5	P0
搜索结果高亮	关键词高亮显示	P1
搜索建议	输入时自动补全	P2
搜索历史	记录最近搜索	P2
搜索结果导出	导出为 Excel	P2

1.3 性能要求

指标	要求
搜索响应	<500ms（10 万条数据）
搜索建议	<100ms
搜索结果	支持分页，每页 20 条
索引更新	实时同步（<1 秒）

二、数据库设计

2.1 FTS5 全文索引

```
-- 案件搜索索引
CREATE VIRTUAL TABLE search_cases USING fts5(
    id,
    name,
    description,
```

```

        tags,
        content='cases',
        content_rowid='rowid'
    );

-- 触发器：自动同步
CREATE TRIGGER cases_ai AFTER INSERT ON cases BEGIN
    INSERT INTO search_cases(rowid, id, name, description, tags)
    VALUES (NEW.rowid, NEW.id, NEW.name, NEW.description, NEW.tags);
END;

CREATE TRIGGER cases_ad AFTER DELETE ON cases BEGIN
    INSERT INTO search_cases(search_cases, rowid, id, name, description, tags)
    VALUES('delete', OLD.rowid, OLD.id, OLD.name, OLD.description, OLD.tags);
END;

CREATE TRIGGER cases_au AFTER UPDATE ON cases BEGIN
    INSERT INTO search_cases(search_cases, rowid, id, name, description, tags)
    VALUES('delete', OLD.rowid, OLD.id, OLD.name, OLD.description, OLD.tags);
    INSERT INTO search_cases(rowid, id, name, description, tags)
    VALUES (NEW.rowid, NEW.id, NEW.name, NEW.description, NEW.tags);
END;

-- 证据文件搜索索引
CREATE VIRTUAL TABLE search_evidence USING fts5(
    id,
    case_id,
    original_name,
    parsed_text,
    tags,
    content='evidence_files',
    content_rowid='rowid'
);

-- 事件节点搜索索引
CREATE VIRTUAL TABLE search_nodes USING fts5(
    id,
    case_id,
    title,
    description,
    tags,
    content='event_nodes',
    content_rowid='rowid'
);

-- 人物搜索索引
CREATE VIRTUAL TABLE search_persons USING fts5(

```

```

        id,
        name,
        name_pinyin,
        organization,
        position,
        phone,
        content='persons',
        content_rowid='rowid'
    );

-- 跨表统一搜索视图
CREATE VIRTUAL TABLE search_all USING fts5(
    id,
    object_type,      -- case/evidence/node/person
    object_id,
    case_id,
    title,
    content,
    tags,
    created_at
);

```

2.2 搜索统计表

```

-- 搜索历史表
CREATE TABLE search_histories (
    id            TEXT PRIMARY KEY,
    user_id       TEXT NOT NULL REFERENCES users(id),
    keyword       TEXT NOT NULL,
    filters       JSON,                -- 搜索条件
    result_count  INTEGER,
    searched_at   DATETIME DEFAULT CURRENT_TIMESTAMP,

    INDEX idx_user ON search_histories(user_id),
    INDEX idx_time ON search_histories(searched_at)
);

-- 热门搜索表
CREATE TABLE hot_searches (
    keyword       TEXT PRIMARY KEY,
    search_count  INTEGER DEFAULT 0,
    last_searched DATETIME,
    updated_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);

```

三、搜索服务实现

3.1 搜索数据结构

```
// src/models/search.rs
use chrono::{DateTime, Utc};
use serde::{Deserialize, Serialize};

/// 搜索对象类型
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
#[serde(rename_all = "lowercase")]
pub enum SearchObjectType {
    Case,
    Evidence,
    Node,
    Person,
}

/// 搜索请求
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct SearchRequest {
    pub keyword: String,
    pub object_types: Option<Vec<SearchObjectType>>,
    pub case_id: Option<String>,
    pub filters: Option<SearchFilters>,
    pub page: i64,
    pub page_size: i64,
}

/// 搜索过滤条件
#[derive(Debug, Clone, Serialize, Deserialize, Default)]
pub struct SearchFilters {
    // 时间范围
    pub start_date: Option<chrono::NaiveDate>,
    pub end_date: Option<chrono::NaiveDate>,

    // 标签
    pub tags: Option<Vec<String>>,

    // 人物
    pub person_ids: Option<Vec<String>>,

    // 文件类型
```

```

    pub file_types: Option<Vec<String>>,

    // 节点类型
    pub node_types: Option<Vec<String>>,

    // 优先级
    pub priorities: Option<Vec<String>>,
}

/// 搜索结果
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct SearchResult {
    pub object_type: SearchObjectType,
    pub object_id: String,
    pub case_id: Option<String>,
    pub case_name: Option<String>,
    pub title: String,
    pub content: Option<String>,
    pub highlight: Option<String>, // 高亮片段
    pub tags: Vec<String>,
    pub created_at: DateTime<Utc>,
    pub score: f64, // 相关度评分
}

/// 搜索响应
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct SearchResponse {
    pub results: Vec<SearchResult>,
    pub total: i64,
    pub page: i64,
    pub page_size: i64,
    pub suggestions: Vec<String>, // 搜索建议
}

```

3.2 搜索服务

```

// src/services/search_service.rs
use sqlx::{SqlitePool, FromRow};
use crate::models::search::*;

pub struct SearchService {
    pool: SqlitePool,
}

impl SearchService {

```

```

pub fn new(pool: SqlitePool) -> Self {
    Self { pool }
}

/// 全局搜索
pub async fn search(
    &self,
    request: &SearchRequest,
) -> anyhow::Result<SearchResponse> {
    let mut results = Vec::new();
    let mut total = 0;

    // 确定搜索范围
    let types_to_search = request.object_types.clone().unwrap_or(vec![
        SearchObjectType::Case,
        SearchObjectType::Evidence,
        SearchObjectType::Node,
        SearchObjectType::Person,
    ]);

    // 分别搜索各类型
    for object_type in &types_to_search {
        let (type_results, type_total) = match object_type {
            SearchObjectType::Case => {
                self.search_cases(&request.keyword, &request.filters, request.page, request.limit)
            }
            SearchObjectType::Evidence => {
                self.search_evidence(&request.keyword, &request.case_id, &request.filters, request.limit)
            }
            SearchObjectType::Node => {
                self.search_nodes(&request.keyword, &request.case_id, &request.filters, request.limit)
            }
            SearchObjectType::Person => {
                self.search_persons(&request.keyword, &request.filters, request.page, request.limit)
            }
        };

        results.extend(type_results);
        total += type_total;
    }

    // 按相关度排序
    results.sort_by(|a, b| b.score.partial_cmp(&a.score).unwrap());

    // 获取搜索建议
    let suggestions = self.get_search_suggestions(&request.keyword).await?;

```

```

// 记录搜索历史
self.record_search_history(request).await?;

Ok(SearchResponse {
    results,
    total,
    page: request.page,
    page_size: request.page_size,
    suggestions,
})
}

/// 搜索案件
async fn search_cases(
    &self,
    keyword: &str,
    filters: &Option<SearchFilters>,
    page: i64,
    page_size: i64,
) -> anyhow::Result<(Vec<SearchResult>, i64)> {
    let offset = (page - 1) * page_size;

    // 构建查询
    let query = format!(
        r#"
        SELECT
            sc.id,
            sc.rowid as object_id,
            c.name as title,
            c.description as content,
            c.tags,
            c.created_at,
            bm25(search_cases) as score
        FROM search_cases sc
        JOIN cases c ON sc.rowid = c.rowid
        WHERE search_cases MATCH ?
        {}
        ORDER BY score DESC
        LIMIT ? OFFSET ?
        "#,
        self.build_filter_clause(filters, "c")
    );

    let mut rows = sqlx::query_as::<_, SearchResultRow>(&query)
        .bind(format!("\"{}\"", keyword))
        .bind(page_size)
        .bind(offset);

```



```

// 绑定过滤条件
self.bind_filters(&mut rows, filters);

let rows = rows.fetch_all(&self.pool).await?;

// 获取总数
let count_query = format!(
    r#"SELECT COUNT(*) FROM search_cases WHERE search_cases MATCH ? {"} "#,
    self.build_filter_clause(filters, "c")
);

let total: (i64,) = sqlx::query_as(&count_query)
    .bind(format!("{}", keyword))
    .fetch_one(&self.pool)
    .await?;

let results = rows.into_iter()
    .map(|r| SearchResult {
        object_type: SearchObjectType::Case,
        object_id: r.object_id,
        case_id: None,
        case_name: None,
        title: r.title,
        content: r.content,
        highlight: self.extract_highlight(&r.content, keyword),
        tags: serde_json::from_str(&r.tags).unwrap_or_default(),
        created_at: r.created_at,
        score: r.score.abs(),
    })
    .collect();

Ok((results, total.0))
}

/// 搜索证据文件
async fn search_evidence(
    &self,
    keyword: &str,
    case_id: &Option<String>,
    filters: &Option<SearchFilters>,
    page: i64,
    page_size: i64,
) -> anyhow::Result<(Vec<SearchResult>, i64)> {
    let offset = (page - 1) * page_size;

    let mut where_clause = String::from("WHERE search_evidence MATCH ?");

```

```

        if let Some(cid) = case_id {
            where_clause.push_str(&format!(" AND se.case_id = '{}'", cid));
        }

        let query = format!(
            r#"
            SELECT
                se.id,
                se.rowid as object_id,
                se.case_id,
                c.name as case_name,
                ef.original_name as title,
                ef.parsed_text as content,
                ef.tags,
                ef.created_at,
                bm25(search_evidence) as score
            FROM search_evidence se
            JOIN evidence_files ef ON se.rowid = ef.rowid
            LEFT JOIN cases c ON se.case_id = c.id
            {}
            ORDER BY score DESC
            LIMIT ? OFFSET ?
            "#,
            where_clause
        );

        let rows = sqlx::query_as:::<_, SearchResultRow>(&query)
            .bind(format!("\"{}\"", keyword))
            .bind(page_size)
            .bind(offset)
            .fetch_all(&self.pool)
            .await?;

        // 获取总数
        let count_query = format!(
            r#"SELECT COUNT(*) FROM search_evidence WHERE search_evidence MATCH ? {}"#,
            where_clause.replace("WHERE", "AND").replace("se.case_id", "case_id")
        );

        let total: (i64,) = sqlx::query_as(&count_query)
            .bind(format!("\"{}\"", keyword))
            .fetch_one(&self.pool)
            .await?;

        let results = rows.into_iter()
            .map(|r| SearchResult {

```

```

        object_type: SearchObjectType::Evidence,
        object_id: r.object_id,
        case_id: Some(r.case_id),
        case_name: r.case_name,
        title: r.title,
        content: r.content,
        highlight: self.extract_highlight(&r.content, keyword),
        tags: serde_json::from_str(&r.tags).unwrap_or_default(),
        created_at: r.created_at,
        score: r.score.abs(),
    })
    .collect();

    Ok((results, total.0))
}

/// 获取搜索建议
async fn get_search_suggestions(&self, keyword: &str) -> anyhow::Result<Vec<String>> {
    if keyword.len() < 2 {
        return Ok(Vec::new());
    }

    // 从热门搜索中获取建议
    let suggestions = sqlx::query_scalar::<_, String>(
        r#"
        SELECT keyword FROM hot_searches
        WHERE keyword LIKE ?
        ORDER BY search_count DESC
        LIMIT 5
        "#,
    )
    .bind(format!("{}", keyword))
    .fetch_all(&self.pool)
    .await?;

    Ok(suggestions)
}

/// 记录搜索历史
async fn record_search_history(&self, request: &SearchRequest) -> anyhow::Result<()> {
    use uuid::Uuid;

    sqlx::query(
        r#"
        INSERT INTO search_histories (id, user_id, keyword, filters, result_count, search_count)
        VALUES (?, ?, ?, ?, ?, ?)
        "#,
    )
    .execute(&self.pool)
    .await?;
}

```

```

        )
        .bind(UUID::new_v4().to_string())
        .bind("current_user_id") // 从上下文获取
        .bind(&request.keyword)
        .bind(serde_json::to_string(&request.filters?))
        .bind(0) // 实际结果数
        .bind(Utc::now())
        .execute(&self.pool)
        .await?;

    Ok(())
}

/// 构建过滤条件
fn build_filter_clause(&self, filters: &Option<SearchFilters>, table_alias: &str) -> String {
    match filters {
        None => String::new(),
        Some(f) => {
            let mut conditions = Vec::new();

            if let Some(start) = &f.start_date {
                conditions.push(format!("{} created_at >= '{}'", table_alias, start));
            }

            if let Some(end) = &f.end_date {
                conditions.push(format!("{} created_at <= '{}'", table_alias, end));
            }

            if conditions.is_empty() {
                String::new()
            } else {
                format!("AND {}", conditions.join(" AND "))
            }
        }
    }
}

/// 搜索结果行（内部使用）
#[derive(Debug, Clone, FromRow)]
struct SearchResultRow {
    id: String,
    object_id: String,
    case_id: Option<String>,
    case_name: Option<String>,
    title: String,
    content: Option<String>,
}

```

```
tags: String,  
created_at: DateTime<Utc>,  
score: f64,  
}
```

四、API 接口设计

4.1 搜索接口

方法	路径	说明
GET	/api/v1/search	全局搜索
GET	/api/v1/search/cases	搜索案件
GET	/api/v1/search/evidence	搜索证据
GET	/api/v1/search/nodes	搜索节点
GET	/api/v1/search/persons	搜索人物
GET	/api/v1/search/suggestions	搜索建议
GET	/api/v1/search/history	搜索历史
DELETE	/api/v1/search/history	清除搜索历史

4.2 搜索接口示例

请求:

```
GET /api/v1/search?keyword=合同&object_types=evidence,node&page=1&page_size=20  
Authorization: Bearer {token}
```

响应:

```

{
  "code": 200,
  "message": "success",
  "data": {
    "results": [
      {
        "object_type": "evidence",
        "object_id": "evidence_001",
        "case_id": "case_123",
        "case_name": "张三诉李四合同纠纷",
        "title": "采购合同.pdf",
        "content": "甲方: XXX 公司\n乙方: XXX 公司...",
        "highlight": "采购<em>合同</em>.pdf",
        "tags": ["合同", "重要"],
        "created_at": "2026-03-14T10:00:00Z",
        "score": 0.95
      },
      {
        "object_type": "node",
        "object_id": "node_001",
        "case_id": "case_123",
        "case_name": "张三诉李四合同纠纷",
        "title": "合同签订",
        "content": "张三与李四签订采购合同",
        "highlight": "<em>合同</em>签订",
        "tags": ["合同"],
        "created_at": "2026-03-14T10:30:00Z",
        "score": 0.85
      }
    ],
    "total": 15,
    "page": 1,
    "page_size": 20,
    "suggestions": ["合同签订", "合同编号", "合同金额"]
  }
}

```

4.3 搜索建议接口

请求:

```
GET /api/v1/search/suggestions?keyword=合
```

响应:

```
{
  "code": 200,
  "data": {
    "suggestions": [
      "合同",
      "合同签订",
      "合同编号",
      "合同金额",
      "合同违约"
    ]
  }
}
```

五、前端实现

5.1 搜索组件

```
// frontend/src/components/SearchBar.rs

#[component]
pub fn SearchBar() -> Element {
  let mut keyword = use_signal(|| String::new());
  let mut suggestions = use_signal(|| Vec::<String>::new());
  let mut show_suggestions = use_signal(|| false);
  let mut is_searching = use_signal(|| false);
  let mut navigate = use_navigator();

  // 获取搜索建议（防抖）
  use_effect(move || {
    let kw = keyword();
    if kw.len() < 2 {
      suggestions.set(Vec::new());
      return;
    }

    let debounce = setTimeout(Duration::from_millis(300), move || {
      spawn(async move {
        let result = fetch_search_suggestions(&kw).await.unwrap();
```

```

        suggestions.set(result);
        show_suggestions.set(true);
    });
});

    move || clearTimeout(debounce)
});

let handle_search = move |_| {
    let kw = keyword();
    if !kw.is_empty() {
        is_searching.set(true);
        navigate.push(&format!("/search?q={}", kw));
    }
};

rsx! {
    div { class: "search-bar",
        div { class: "search-input-wrapper",
            input {
                r#type: "text",
                placeholder: "搜索案件、证据、节点、人物...",
                value: keyword(),
                oninput: move |e| {
                    keyword.set(e.value());
                    show_suggestions.set(true);
                },
                onkeydown: move |e| {
                    if e.key() == "Enter" {
                        handle_search();
                    }
                },
                onfocus: move |_| show_suggestions.set(true),
                onblur: move |_| {
                    setTimeout(Duration::from_millis(200), move || {
                        show_suggestions.set(false);
                    });
                },
            },
        },
    }

    button {
        class: "search-btn",
        onclick: handle_search,
        disabled: is_searching(),
        if is_searching() { "🔍" } else { "搜索" }
    }
}

```



```

// 搜索建议下拉框
if show_suggestions() && !suggestions().is_empty() {
  div { class: "search-suggestions",
    for suggestion in suggestions().iter() {
      div {
        class: "suggestion-item",
        onclick: move |_| {
          keyword.set(suggestion.clone());
          handle_search(());
        },
        {suggestion}
      }
    }
  }
}
}
}
}
}
}
}
}
}
}

```

5.2 搜索结果页面

```

// frontend/src/pages/SearchResults.rs

#[component]
pub fn SearchResultsPage() -> Element {
  let query = use_query::<SearchQuery>();
  let search_results = use_resource(move || {
    let q = query.clone();
    async move {
      search_api(&q.keyword, &q.object_types, q.page, q.page_size).await
    }
  });

  rsx! {
    div { class: "search-results-page",
      h1 { "搜索结果: {query.keyword}" }

      // 搜索类型筛选
      div { class: "search-filters",
        label {
          input { r#type: "checkbox", checked: true, "全部" }
        }
        label {
          input { r#type: "checkbox", checked: true, "证据" }

```

```

    }
    label {
        input { r#type: "checkbox", checked: true, "节点" }
    }
    label {
        input { r#type: "checkbox", checked: true, "人物" }
    }
}

match &*search_results.value() {
    Some(Ok(response)) => rsx! {
        div { class: "search-summary",
            "共找到 {response.total} 条结果"
        }

        // 按类型分组显示
        for (object_type, results) in group_by_type(&response.results) {
            div { class: "result-group",
                h3 { "{get_type_name(&object_type)} ({results.len()})" }

                for result in results {
                    SearchResultItem { result: result.clone() }
                }
            }
        }

        // 分页
        Pagination {
            current_page: response.page,
            total_pages: (response.total / response.page_size) + 1,
            on_page_change: move |page| {
                query.page = page;
                search_results.restart();
            },
        },
    },
    Some(Err(e)) => rsx! {
        div { class: "error", "搜索失败: {e}" }
    },
    None => rsx! {
        div { class: "loading", "搜索中..." }
    },
}
}
}
}
}

```

```

#[component]
fn SearchResultItem(result: SearchResult) -> Element {
  rsx! {
    div { class: "search-result-item",
      div { class: "result-header",
        span { class: "result-type", "{get_type_icon(&result.object_type)}" }
        h4 { class: "result-title",
          Link {
            to: get_result_link(&result),
            dangerous_inner_html: "{result.highlight.unwrap_or(result.title)}"
          }
        }
      }
      if let Some(case_name) = &result.case_name {
        span { class: "result-case", "· {case_name}" }
      }
    }

    if let Some(content) = &result.content {
      p { class: "result-content",
        dangerous_inner_html: "{extract_snippet(content, &result.keyword)}"
      }
    }

    div { class: "result-meta",
      if !result.tags.is_empty() {
        span { class: "result-tags",
          for tag in result.tags.iter().take(3) {
            span { class: "tag", {tag} }
          }
        }
      }
      span { class: "result-date", "{format_date(result.created_at)}" }
      span { class: "result-score", "相关度: {(result.score * 100) as i32}%" }
    }
  }
}

```

六、性能优化

6.1 索引优化

```
-- 优化 FTS5 配置
CREATE VIRTUAL TABLE search_evidence USING fts5(
    original_name,
    parsed_text,
    tags,
    content='evidence_files',
    content_rowid='rowid',
    tokenize='unicode61' -- 支持中文分词
);

-- 定期优化索引
OPTIMIZE search_evidence;

-- 分析索引统计信息
ANALYZE search_evidence;
```

6.2 查询优化

```
// 使用 LIMIT 限制结果数
pub async fn search_with_limit(
    keyword: &str,
    limit: i64,
) -> Vec<SearchResult> {
    sqlx::query_as(
        "SELECT * FROM search_evidence
        WHERE search_evidence MATCH ?
        ORDER BY bm25(search_evidence)
        LIMIT ?",
    )
    .bind(keyword)
    .bind(limit)
    .fetch_all(&pool)
    .await
}

// 缓存热门搜索结果
use moka::future::Cache;
```

```
let cache = Cache::new(1000);

pub async fn search_with_cache(
    keyword: String,
) -> Vec<SearchResult> {
    cache.get_or_insert_with(keyword.clone(), || async move {
        search_without_cache(&keyword).await
    }).await
}
```

6.3 前端优化

优化项	说明
防抖	搜索建议 300ms 防抖
虚拟滚动	大量结果时只渲染可见区域
结果缓存	相同搜索词缓存结果
分页加载	每页 20 条，滚动加载更多

七、总结

7.1 技术选型

模块	技术选型	说明
全文搜索	SQLite FTS5	零依赖、够用
分词	unicode61	支持中文分词
相关度排序	BM25 算法	FTS5 内置
搜索建议	热门搜索表	统计高频搜索词

7.2 API 清单

接口	方法	说明
/api/v1/search	GET	全局搜索
/api/v1/search/cases	GET	搜索案件
/api/v1/search/evidence	GET	搜索证据
/api/v1/search/nodes	GET	搜索节点
/api/v1/search/persons	GET	搜索人物
/api/v1/search/suggestions	GET	搜索建议

7.3 性能指标

指标	目标	实测
搜索响应	<500ms	~200ms
搜索建议	<100ms	~50ms
索引同步	<1 秒	~500ms
支持数据量	10 万条	10 万 +

文档版本 : v1.0

最后更新: 2026 年 3 月 14 日

开发负责人: 王舟

邮箱 : main@mails.wedevs.org

电话: 15378391447

开发负责人：王舟 | 电话：15378391447 | 邮箱：main@mails.wedevs.org